

COMPATIBLE BASES AND ACTUATOR SCHEDULING

DAWID E. KĘDZIERSKI AND PIOTR KRASOŃ

ABSTRACT. We study a constrained basis-selection problem for an ordered block matrix $B = [B_1 \mid \cdots \mid B_n]$. The successive tail matrices determine a nested family of column spaces, and we consider bases of representatives which are compatible with this induced flag. Our main linear-algebraic result shows that, among bases of representatives, compatibility with the entire flag is characterized by the intrinsic block quotas $r_k = w_k - w_{k+1}$, where w_k is the rank of the k -th tail matrix.

We then show that the same structure arises naturally in finite-horizon linear control. For the reachability matrix $[A^{N-1}G \mid \cdots \mid G]$, requiring every actuator to be used exactly once turns an actuator schedule into a representative system. Preservation of the complete reachability profile is then equivalent to compatibility with the tail-space flag.

This correspondence leads to an exact depth-first search with a future-rank pruning criterion. For an explicit structured family and a fixed search ordering, the baseline search visits $3 \cdot 2^m + m - 2$ nodes before the first solution, whereas future-rank pruning reduces this number to $3m + 1$. Exact-arithmetic experiments reproduce these formulas for all tested instances.

1. INTRODUCTION

Consider an ordered block matrix

$$B = [B_1 \mid \cdots \mid B_n].$$

For each column position, the corresponding columns of the blocks form a family of possible representatives. The classical problem is to choose one vector from each family so that the selected vectors are linearly independent, as in the theory of independent transversals and Rado's theorem [9]. Related representative-selection and matroid structures are discussed in [2, 3, 8, 10].

The ordered block structure leads naturally to the tail matrices

$$B_{\geq k} = [B_k \mid \cdots \mid B_n]$$

and their nested column spaces

$$W_n \subseteq W_{n-1} \subseteq \cdots \subseteq W_1.$$

We study bases of representatives compatible with this flag, meaning that for every k , the representatives selected from $B_k, \dots, B_n$ span W_k .

Let

$$w_k = \dim W_k, \quad r_k = w_k - w_{k+1},$$

2020 *Mathematics Subject Classification.* 15A03, 93B03, 93B05.

Key words and phrases. compatible bases, nested subspaces, rank profiles, representative systems, actuator scheduling, controllability.

with $w_{n+1} = 0$. Our main linear-algebraic result shows that, among bases of representatives, compatibility with the whole flag is equivalent to the simple block-count condition that exactly r_k vectors are selected from B_k .

This structure has a direct interpretation in finite-horizon linear control. For

$$x_{t+1} = Ax_t + Gu_t,$$

the reachability matrix

$$\begin{bmatrix} A^{N-1}G & A^{N-2}G & \cdots & G \end{bmatrix}$$

has the same block form. If every actuator is used exactly once, choosing its activation time amounts to choosing one representative from the corresponding column family. We show that schedules preserving the complete tail reachability profile are precisely the compatible bases above. In particular, this requirement is stronger than terminal controllability. Related questions of controllability, sparse actuator scheduling, and actuator placement have been studied in [1, 4, 5, 6, 11, 12].

The same characterization yields an exact depth-first search. In addition to block quotas and linear independence, a future-rank test rejects partial selections that can no longer attain the required tail ranks. For an explicit structured family and a fixed search ordering, the baseline search visits

$$3 \cdot 2^m + m - 2$$

nodes before the first solution, whereas future-rank pruning reduces this number to

$$3m + 1.$$

Exact-arithmetic experiments reproduce these formulas for all tested instances.

2. RECTANGULAR TAIL MATRICES AND RANK PROFILES

Let L be a field and let $V = L^d$. Consider a block matrix $B = \begin{bmatrix} B_1 & B_2 & \cdots & B_n \end{bmatrix}$, where $B_j \in \text{Mat}_d(L)$. For each $j = 1, \dots, n$, write $B_j = \begin{bmatrix} b_j^1 & b_j^2 & \cdots & b_j^d \end{bmatrix}$, where $b_j^i \in V$ denotes the i -th column of B_j .

Definition 2.1. For $k = 1, \dots, n$, the *tail matrix* of level k is

$$B_{\geq k} := \begin{bmatrix} B_k & B_{k+1} & \cdots & B_n \end{bmatrix}.$$

Thus $B_{\geq k} \in \text{Mat}_{d \times d(n-k+1)}(L)$. In particular, the natural matrices governing the problem are rectangular, even though the individual blocks B_j are square.

Set $w_k := \text{rank } B_{\geq k}$ for $k = 1, \dots, n$, and $w_{n+1} := 0$. Then $0 = w_{n+1} \leq w_n \leq \cdots \leq w_1 \leq d$.

Definition 2.2. The sequence $(w_1, \dots, w_n)$ is called the *tail rank profile* of B .

For $k = 1, \dots, n$, put $r_k := w_k - w_{k+1}$. Then $r_k \geq 0$ and $\sum_{k=1}^n r_k = w_1 = \text{rank } B$. In particular, if B has full row rank, then $\sum_{k=1}^n r_k = d$.

For $k = 1, \dots, n$, let $W_k := \text{Col}(B_{\geq k})$ and set $W_{n+1} := \{0\}$. Equivalently, $W_k = \text{Span} \{b_j^i : j \geq k, 1 \leq i \leq d\}$. Since $B_{\geq k+1}$ is obtained from $B_{\geq k}$ by deleting the block B_k , these spaces form the flag

$$(2.1) \quad \{0\} = W_{n+1} \subseteq W_n \subseteq W_{n-1} \subseteq \dots \subseteq W_1 \subseteq V.$$

Moreover, $\dim W_k = w_k$, and hence

$$r_k = \dim W_k - \dim W_{k+1} = \dim (W_k / W_{k+1}).$$

Thus r_k measures the increase in dimension contributed at level k relative to the subsequent tail.

3. COMPATIBLE BASES OF REPRESENTATIVES

For each column position $i = 1, \dots, d$, consider the indexed family

$$\mathcal{V}^i = \{(j, b_j^i) : j = 1, \dots, n\}.$$

Thus the block index from which a representative is selected is regarded as part of the selection data. A representative system chooses one indexed vector $(j_i, b_{j_i}^i)$ from each family $\mathcal{V}^i$.

Definition 3.1. A basis $\mathcal{B} = (b_{j_1}^1, \dots, b_{j_d}^d)$ of V is called a *basis of representatives compatible with the flag* (2.1) if, for every $k = 1, \dots, n$,

$$\text{Span}(\mathcal{B}_{\geq k}) = W_k, \quad \mathcal{B}_{\geq k} := \{b_{j_s}^s \in \mathcal{B} : j_s \geq k\}.$$

We always regard the indices $j_1, \dots, j_d$ as part of the data of the representative system. The tail rank profile determines automatically how many representatives must come from each block.

Proposition 3.1. Let $\mathcal{B} = (b_{j_1}^1, \dots, b_{j_d}^d)$ be a basis of representatives compatible with the flag. Then, for every $k = 1, \dots, n$,

$$\#\{i : j_i = k\} = w_k - w_{k+1} = r_k.$$

Proof. Since $\mathcal{B}$ is a basis, every subfamily is linearly independent. Compatibility therefore gives $\#\mathcal{B}_{\geq k} = \dim W_k = w_k$ and $\#\mathcal{B}_{\geq k+1} = w_{k+1}$. Their difference is precisely the number of vectors selected from the k -th block, which proves the assertion. $\square$

Proposition 3.2 (Quota characterization). Let $\mathcal{B} = (b_{j_1}^1, \dots, b_{j_d}^d)$ be a basis of representatives. Then $\mathcal{B}$ is compatible with the tail-space flag if and only if

$$\#\{i : j_i = k\} = r_k, \quad k = 1, \dots, n.$$

Proof. Necessity follows from Proposition 3.1.

Conversely, suppose that $\#\{i : j_i = k\} = r_k$ for every k . Fix k . Then

$$\#\mathcal{B}_{\geq k} = \sum_{j=k}^n r_j = w_k.$$

Since $\mathcal{B}_{\geq k}$ is a subfamily of the basis $\mathcal{B}$, it is linearly independent, and hence $\dim \text{Span}(\mathcal{B}_{\geq k}) = w_k$.

Every vector in $\mathcal{B}_{\geq k}$ belongs to one of the blocks $B_k, \dots, B_n$. Hence

$$\text{Span}(\mathcal{B}_{\geq k}) \subseteq W_k.$$

Since both spaces have dimension w_k , they are equal. Thus $\text{Span}(\mathcal{B}_{\geq k}) = W_k$ for every k , and $\mathcal{B}$ is compatible with the tail-space flag. $\square$

Remark 3.2. The numbers r_k are determined entirely by the matrix B . They specify how many representatives a compatible basis must select from each block B_k . Conversely, once a representative system is known to be a basis, these block counts are sufficient for compatibility: it is compatible with the tail-space flag if and only if it selects exactly r_k vectors from B_k for every k .

Assume now that $\text{rank } B = d$, and set

$$k_0 := \max\{k : w_k = d\}.$$

Proposition 3.3 (Trimming). *Let $k_0 = \max\{k : w_k = d\}$. Then $r_1 = \dots = r_{k_0-1} = 0$. Consequently, no compatible basis of representatives contains a vector from $B_1, \dots, B_{k_0-1}$, and the compatible-basis problem for B is equivalent to the same problem for the trimmed matrix*

$$B_{\text{trim}} := B_{\geq k_0}.$$

Proof. By definition of k_0 , $w_1 = \dots = w_{k_0} = d$, so $r_j = w_j - w_{j+1} = 0$ for every $j < k_0$. By Proposition 3.2, a compatible basis therefore uses no representatives from $B_1, \dots, B_{k_0-1}$. These blocks may consequently be deleted without changing either the existence or the set of compatible bases. $\square$

Thus B_{trim} is the shortest tail of B having full row rank.

Problem 3.4 (Compatible representative basis). *Given a block matrix B of full row rank, determine whether it admits a basis of representatives compatible with its tail-space flag, and construct one whenever it exists.*

4. SINGLE-USE ACTUATOR SCHEDULING

We now show how the compatible-basis problem arises naturally in a finite-horizon control setting.

Consider the discrete-time linear system

$$(4.1) \quad x_{t+1} = Ax_t + Gu_t, \quad t = 0, \dots, N-1,$$

where $A \in \mathbb{R}^{d \times d}$ and $G = [g_1, \dots, g_d] \in \mathbb{R}^{d \times d}$. We write $u_t = (u_t^1, \dots, u_t^d)^T$.

The matrix A describes the evolution of the state during one time step. The matrix G is the actuator matrix: its i -th column g_i is the state direction directly affected by actuator i , while u_t^i is the scalar input applied to this actuator at time t .

For reachability we take $x_0 = 0$. Iterating (4.1) gives

$$x_N = \sum_{j=1}^N A^{N-j} G u_{j-1}.$$

We call j the j -th *control slot*. Thus slot $j = 1$ corresponds to time $t = 0$, whereas slot $j = N$ corresponds to the last control action, at time $t = N - 1$.

If actuator i is used in slot j with scalar amplitude α , its contribution to the terminal state is

$$\alpha A^{N-j} g_i.$$

Hence an actuator used earlier is propagated through more powers of A : in the first slot its terminal direction is $A^{N-1} g_i$, whereas in the last slot it is simply g_i .

For each slot j , define

$$(4.2) \quad B_j := A^{N-j} G.$$

Thus the i -th column of B_j is $A^{N-j} g_i$, the terminal direction produced by using actuator i in slot j . The full finite-horizon reachability matrix is therefore

$$B = [A^{N-1} G \mid A^{N-2} G \mid \cdots \mid G].$$

In this sense, B is a catalogue of all actuator–slot terminal directions available over the horizon.

For $k = 1, \dots, N$, the tail space

$$W_k = \text{Col} [A^{N-k} G \mid A^{N-k-1} G \mid \cdots \mid G]$$

has a direct control interpretation: it is the space of terminal-state increments that can still be generated when only the control slots $k, k+1, \dots, N$ remain available. Thus W_1 is the finite-horizon reachable space, whereas $W_N = \text{Col}(G)$. We write $w_k = \dim W_k$ and assume throughout this section that $w_1 = d$. Equivalently, the system is reachable over the prescribed horizon N .

4.1. Single-use schedules. We now impose the restriction that every actuator is used exactly once during the horizon.

Definition 4.1. A *single-use actuator schedule* is a map

$$\tau : \{1, \dots, d\} \longrightarrow \{1, \dots, N\},$$

where $\tau(i)$ is the control slot in which actuator i is used. Different actuators may be assigned to the same slot.

The schedule specifies when an actuator is used, but not its scalar amplitude. If actuator i is assigned amplitude α_i , then the input in slot j is

$$u_{j-1} = \sum_{\tau(i)=j} \alpha_i e_i,$$

where e_i is the i -th standard basis vector. Consequently, the terminal state generated by the schedule is

$$x_N = \sum_{i=1}^d \alpha_i A^{N-\tau(i)} g_i.$$

Thus the schedule selects the d terminal directions

$$(4.3) \quad \mathcal{B}(\tau) = (A^{N-\tau(1)} g_1, \dots, A^{N-\tau(d)} g_d) = (b_{\tau(1)}^1, \dots, b_{\tau(d)}^d).$$

Definition 4.2. A single-use schedule τ is called *terminally controllable* if the selected directions span $\mathbb{R}^d$, equivalently,

$$\text{rank} [A^{N-\tau(1)}g_1 \ \dots \ A^{N-\tau(d)}g_d] = d.$$

Terminal controllability concerns only the state that can be reached at the end of the horizon. We shall require more: the schedule should not exhaust the available control directions too early.

At the beginning of slot k , the actuators with $\tau(i) < k$ have already been used, while those with $\tau(i) \geq k$ are still available. Their remaining terminal directions span

$$\text{Span} \{A^{N-\tau(i)}g_i : \tau(i) \geq k\}.$$

If no single-use schedule is imposed, the terminal directions available from slots $k, \dots, N$ span W_k .

Definition 4.3. A single-use actuator schedule τ is called *reachability-profile preserving* if

$$(4.4) \quad \text{Span} \{A^{N-\tau(i)}g_i : \tau(i) \geq k\} = W_k$$

for every $k = 1, \dots, N$.

Thus a profile-preserving schedule retains at every stage exactly the same future terminal reachability space as the unrestricted system. For $k = 1$, this condition gives $W_1 = \mathbb{R}^d$, so every profile-preserving schedule is terminally controllable.

Example 4.1. Let $d = 2$, $N = 3$, and consider

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad G = I_2 = [g_1, g_2].$$

Thus $g_1 = (1, 0)^T$ and $g_2 = (0, 1)^T$.

The two columns of G describe the instantaneous directions of the two actuators. The matrix A describes how these directions evolve between consecutive control slots. In particular, $Ag_1 = (1, 1)^T$ and $Ag_2 = (0, 1)^T$.

Since $N = 3$, the three slot matrices are

$$B_1 = A^2G = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad B_2 = AG = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad B_3 = G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The columns of B_1 are the terminal effects of firing the two actuators in the first slot, those of B_2 correspond to the second slot, and those of B_3 to the final slot. Thus

$$B = [B_1 \mid B_2 \mid B_3] = [A^2G \mid AG \mid G]$$

collects all possible actuator–slot choices.

Consider the schedule

$$\tau(1) = 1, \quad \tau(2) = 3.$$

It selects the first column of B_1 and the second column of B_3 , namely $A^2g_1 = (1, 2)^T$ and $g_2 = (0, 1)^T$. With amplitudes α_1, α_2 ,

$$x_3 = \alpha_1(1, 2)^T + \alpha_2(0, 1)^T.$$

These two directions are linearly independent, so the schedule is terminally controllable.

It is not, however, reachability-profile preserving. At the beginning of slot 2, actuator 1 has already been used, and only actuator 2 remains available. Hence the remaining selected directions span only $\text{Span}\{g_2\}$. On the other hand, the unrestricted system still has both actuators available in slots 2 and 3, and $W_2 = \mathbb{R}^2$. Therefore

$$\text{Span}\{g_2\} \neq W_2.$$

By contrast, the schedule

$$\tau(1) = \tau(2) = 3$$

keeps both actuators available until the last slot. Their terminal directions are then g_1 and g_2 , and the schedule preserves the complete reachability profile.

Remark 4.4. A concrete physical picture is an idealized small satellite equipped with two one-shot thrusters. The columns of G describe the instantaneous state changes produced by the two thrusters, while A describes how these changes propagate through the subsequent dynamics.

Firing a thruster in an early slot may strongly influence the terminal state because its effect is propagated for several time steps. Once fired, however, that thruster is no longer available for later corrections. In Example 4.1, firing actuator 1 in the first slot still allows full terminal controllability, but it removes one control direction from the remaining part of the horizon. The profile-preserving condition prevents this premature loss of future control authority.

4.2. Equivalence with compatible bases. The preceding construction identifies the scheduling problem directly with the representative-basis problem.

Theorem 4.1. *Let $B_j = A^{N-j}G$. There is a one-to-one correspondence between reachability-profile preserving single-use actuator schedules and bases of representatives compatible with the tail-space flag of B . The correspondence is*

$$\tau \longmapsto \mathcal{B}(\tau) = (b_{\tau(1)}^1, \dots, b_{\tau(d)}^d).$$

Proof. A schedule chooses, for each actuator i , exactly one indexed vector $b_{\tau(i)}^i$, and hence determines a representative system.

If the schedule is profile preserving, the condition for $k = 1$ shows that the selected d vectors span $\mathbb{R}^d$, and hence form a basis. For every k , the vectors corresponding to actuators scheduled in slots $k, \dots, N$ are precisely $\{b_{\tau(i)}^i : \tau(i) \geq k\}$. Therefore condition (4.4) is exactly the compatibility condition with the tail space W_k . The converse is identical. $\square$

The quota characterization of Section 3 now gives a simple description of all profile-preserving schedules.

Corollary 4.2 (Schedule characterization). *A single-use actuator schedule τ is reachability-profile preserving if and only if*

(i) it is terminally controllable, that is,

$$\text{rank} \begin{bmatrix} A^{N-\tau(1)}g_1 & \cdots & A^{N-\tau(d)}g_d \end{bmatrix} = d;$$

(ii) exactly $r_k = w_k - w_{k+1}$ actuators are assigned to slot k , that is,

$$\#\{i : \tau(i) = k\} = r_k, \quad k = 1, \dots, N.$$

Proof. By Theorem 4.1, the schedule corresponds to the representative system $\mathcal{B}(\tau)$. Condition (i) says that this system is a basis, while condition (ii) gives the block quotas. The result follows from Proposition 3.2. $\square$

Thus r_k is exactly the number of actuators that a reachability-profile preserving schedule must assign to control slot k .

5. PREPROCESSING AND CANDIDATE CONSTRUCTION

We first reduce the search to the shortest tail having full row rank. Starting from $w_{n+1} = 0$, we compute the tail ranks

$$w_j = \text{rank } B_{\geq j}, \quad j = n, n-1, \dots, 1,$$

and the corresponding block quotas

$$r_j = w_j - w_{j+1}.$$

If $\text{rank } B < d$, no compatible basis exists. Otherwise, let

$$k_0 = \max \{k : w_k = d\}.$$

By Proposition 3.3, only the blocks $B_{k_0}, \dots, B_n$ need to be considered.

For each representative family $i = 1, \dots, d$, define the candidate list

$$L_i = \{(i, j, b_j^i) : k_0 \leq j \leq n, b_j^i \neq 0\}.$$

Within each list, candidates are considered in decreasing block index. If some L_i is empty, no basis of representatives exists.

The candidate lists are ordered by increasing cardinality,

$$\mathbf{OL} = (L_{i_1}, \dots, L_{i_d}), \quad \#L_{i_1} \leq \dots \leq \#L_{i_d},$$

with ties resolved by preserving the original order. This is a fail-first ordering: representative families with fewer admissible candidates are processed earlier.

5.1. Baseline depth-first search. We retain the basic depth-first search as a reference algorithm. At each level it checks whether the block quota has been exhausted and whether the new vector is independent of the previously selected vectors.

Algorithm 1 Baseline depth-first selection

```

1: procedure DFSSELECTION(OL, Required,  $k_0, n$ )
2:    $d \leftarrow \# \mathbf{OL}$ 
3:   Basis  $\leftarrow$  (None, ..., None)
4:   Selected  $\leftarrow$  (None, ..., None)
5:   for  $j = k_0, \dots, n$  do
6:      $\text{Used}[j] \leftarrow 0$ 
7:   procedure DFS( $\ell$ )
8:     if  $\ell > d$  then
9:       return SUCCESS
10:    for all  $(i, m, c) \in \mathbf{OL}[\ell]$  do
11:      if  $\text{Used}[m] \geq \mathbf{Required}[m]$  then
12:        continue
13:      if  $c \notin \text{Span}(\mathbf{Basis}[1 : \ell - 1])$  then
14:         $\mathbf{Basis}[\ell] \leftarrow c$ 
15:         $\mathbf{Selected}[\ell] \leftarrow (i, m, c)$ 
16:         $\text{Used}[m] \leftarrow \text{Used}[m] + 1$ 
17:         $result \leftarrow \text{DFS}(\ell + 1)$ 
18:        if  $result = \text{SUCCESS}$  then
19:          return SUCCESS
20:           $\text{Used}[m] \leftarrow \text{Used}[m] - 1$ 
21:           $\mathbf{Basis}[\ell] \leftarrow \text{None}$ 
22:           $\mathbf{Selected}[\ell] \leftarrow \text{None}$ 
23:        return FAILURE
24:   $result \leftarrow \text{DFS}(1)$ 
25:  if  $result = \text{SUCCESS}$  then
26:    return (True, Selected)
27:  else
28:    return (False,  $\emptyset$ )

```

▷ Backtrack

Here $\mathbf{Required}[k] = r_k$.

The complete baseline procedure consists of the preprocessing and candidate construction described above, followed by DFSSELECTION. If the search succeeds, the selected triples (i, m, c) determine the compatible basis by placing c in representative position i .

6. REACHABILITY-AWARE PRUNING

The baseline search uses only local information: block quotas and linear independence. We now use the tail spaces to detect partial selections that cannot be completed.

Let S be a partial selection. For each k , let $S_{\geq k}$ denote the vectors already selected from blocks $B_k, \dots, B_n$. Let $I(S)$ be the set of unassigned representative families, and let $u_j(S)$ be the number of vectors already selected from B_j .

For $k = 1, \dots, n$, define

$$R_{\geq k}(S) = \{b_j^i : i \in I(S), j \geq k, u_j(S) < r_j, b_j^i \neq 0\}.$$

Thus $R_{\geq k}(S)$ collects all candidates that are individually admissible for extending S within the tail $B_k, \dots, B_n$. Since it may contain several candidates from the same unassigned representative family, its span can be larger than the space attainable by an actual completion.

Example 6.1. Let $d = 4$, $n = 3$, with

$$(r_1, r_2, r_3) = (1, 1, 2), \quad (w_1, w_2, w_3) = (4, 3, 2),$$

and

$$B_1 = [0, 0, e_4, 0], \quad B_2 = [e_3, 0, e_1 + e_2, e_2], \quad B_3 = [e_1, e_2, 0, 0].$$

For the partial selection

$$S = \{b_3^1, b_3^2\} = \{e_1, e_2\},$$

we have $I(S) = \{3, 4\}$ and $u_3(S) = 2 = r_3$. Hence, at level $k = 2$,

$$R_{\geq 2}(S) = \{b_2^3, b_2^4\} = \{e_1 + e_2, e_2\}.$$

Therefore

$$\dim \text{Span}(S_{\geq 2} \cup R_{\geq 2}(S)) = 2 < 3 = w_2,$$

so the partial selection cannot be completed.

6.1. Future-rank feasibility.

Proposition 6.1 (Future-rank pruning). *If, for some k ,*

$$(6.1) \quad \dim \text{Span}(S_{\geq k} \cup R_{\geq k}(S)) < w_k,$$

then S cannot be extended to a compatible basis.

Proof. For any completion $\mathcal{B}$ of S ,

$$\mathcal{B}_{\geq k} \subseteq S_{\geq k} \cup R_{\geq k}(S).$$

Hence

$$\dim \text{Span}(\mathcal{B}_{\geq k}) \leq \dim \text{Span}(S_{\geq k} \cup R_{\geq k}(S)).$$

If the latter dimension is smaller than $w_k = \dim W_k$, then $\text{Span}(\mathcal{B}_{\geq k}) \neq W_k$. $\square$

Remark 6.1. The future-rank condition is necessary but not sufficient for extendability, since $R_{\geq k}(S)$ may contain several candidates from the same unassigned representative family. Thus

$$\dim \text{Span}(S_{\geq k} \cup R_{\geq k}(S))$$

is only an upper bound on the tail rank attainable by a completion.

In the control interpretation, a partial schedule may therefore be discarded whenever the already selected and still admissible actuator–slot directions cannot span one of the required tail reachability spaces W_k .

6.2. Residual capacity. The remaining block quotas give another necessary condition. Define

$$q_j(S) := r_j - u_j(S),$$

and, for $J \subseteq \{k_0, \dots, n\}$, let $\Gamma_S(J)$ be the set of unassigned representative families having an admissible candidate in a block belonging to J . Any extendable partial selection satisfies

$$(6.2) \quad \sum_{j \in J} q_j(S) \leq |\Gamma_S(J)| \quad \text{for every } J \subseteq \{k_0, \dots, n\}.$$

This is a Hall-type residual capacity condition. Equivalently, residual capacity may be tested by a bipartite matching or maximum-flow problem. It provides an additional safe pruning criterion.

6.3. Future-feasibility procedure. The implemented feasibility test combines the block-capacity check with the future-rank condition.

Algorithm 2 Future-feasibility test

```

1: procedure FUTUREFEASIBLE(Selected, Used, OL, Flag, Required,  $k_0, n$ )
2:   for  $j = k_0, \dots, n$  do
3:     if Used[ $j$ ] > Required[ $j$ ] then
4:       return False
5:   for  $k = n, n-1, \dots, k_0$  do
6:     construct  $S_{\geq k}$  from the selected triples
7:     construct  $R_{\geq k}(S)$  from the unassigned candidate lists
8:     if  $\text{rank}(S_{\geq k} \cup R_{\geq k}(S)) < \text{Flag}[k]$  then
9:       return False
10:  return True

```

Here **Flag**[k] = w_k and **Required**[k] = r_k .

6.4. Modified depth-first search. The reachability-aware search is obtained from the baseline search by applying Algorithm 2 after every tentative selection. Recursion proceeds only if the enlarged partial selection passes this test; otherwise the algorithm backtracks immediately. All candidate ordering, quota, independence, and backtracking steps are unchanged.

Thus the baseline and modified searches differ only by the future-feasibility test. For the control problem, a selected triple (i, j, c) assigns actuator i to slot j , so the resulting schedule is given by

$$\tau(i) = j.$$

7. CORRECTNESS

We now verify that neither the baseline search nor the reachability-aware pruning changes the set of solutions.

Theorem 7.1. *The baseline algorithm and the modified algorithm return a compatible basis if and only if such a basis exists.*

Proof. Suppose first that either algorithm returns a complete selection. Exactly one vector has then been chosen from each representative family, and the independence test guarantees that the resulting d vectors form a basis.

For every block B_j , the search enforces the upper bound r_j . Since

$$\sum_{j=k_0}^n r_j = d,$$

a complete selection of d vectors satisfying these bounds must use exactly r_j vectors from each block B_j . By Proposition 3.2, the resulting basis is therefore compatible with the tail-space flag.

Conversely, suppose that a compatible basis $\mathcal{B}$ exists. By Proposition 3.3, it uses only the blocks $B_{k_0}, \dots, B_n$, so all of its vectors occur in the candidate lists considered by the search. Hence $\mathcal{B}$ determines a branch of the search tree.

No partial selection along this branch is rejected by the independence test, since every subfamily of a basis is linearly independent. Moreover, Proposition 3.1 shows that the branch never exceeds any block quota.

It remains to consider the additional future-rank test used by the modified algorithm. Let S be a partial selection lying on the branch determined by $\mathcal{B}$. For any k , every vector of $\mathcal{B}_{\geq k}$ is either already contained in $S_{\geq k}$ or belongs to an unassigned representative family in a block whose quota has not yet been filled. Hence it is contained in $R_{\geq k}(S)$. Therefore

$$\mathcal{B}_{\geq k} \subseteq S_{\geq k} \cup R_{\geq k}(S).$$

Since $\mathcal{B}$ is compatible, $\text{Span}(\mathcal{B}_{\geq k}) = W_k$, and consequently

$$\dim \text{Span}(S_{\geq k} \cup R_{\geq k}(S)) \geq w_k.$$

Thus the future-rank test never rejects this branch.

The branch corresponding to $\mathcal{B}$ therefore remains in the finite search tree and is eventually reached by both algorithms. $\square$

Combining Theorem 7.1 with Theorem 4.1 gives the corresponding statement for the control problem.

Corollary 7.2. *For $B = [A^{N-1}G \mid A^{N-2}G \mid \dots \mid G]$, the modified algorithm returns a solution if and only if there exists a reachability-profile preserving single-use actuator schedule.*

8. COMPLEXITY AND SEARCH REDUCTION

Let $N_i = \#L_i$. After ordering the candidate lists as

$$\mathbf{OL} = (L_{i_1}, \dots, L_{i_d}), \quad N_{i_1} \leq \dots \leq N_{i_d},$$

the search tree has depth d . Without pruning, the number of nodes at depth ℓ is at most

$$\prod_{q=1}^{\ell} N_{i_q},$$

and hence the total number of recursive nodes is bounded by

$$1 + \sum_{\ell=1}^d \prod_{q=1}^{\ell} N_{i_q}.$$

Thus the exact search remains combinatorial in the worst case.

The baseline search prunes only when a block quota is exhausted or a linear dependence occurs. The future-rank test may reject a branch earlier whenever

$$\dim \text{Span}(S_{\geq k} \cup R_{\geq k}(S)) < w_k$$

for some k . This does not improve the general worst-case bound, but can substantially reduce the number of visited nodes for structured instances. Section 10 gives an explicit family exhibiting such a reduction.

9. A THREE-DIMENSIONAL EXAMPLE

Consider

$$A_0 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 0 \end{bmatrix}, \quad G_0 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad N = 3.$$

Writing $G_0 = [g_1, g_2, g_3]$, the reachability blocks are

$$B_1 = A_0^2 G_0 = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & -1 \end{bmatrix}, \quad B_2 = A_0 G_0 = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & 1 & 0 \end{bmatrix}, \quad B_3 = G_0 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Their tail-rank and quota profiles are

$$(w_1, w_2, w_3) = (3, 3, 2), \quad (r_1, r_2, r_3) = (0, 1, 2).$$

The schedule

$$\tau_{\text{good}} = (3, 2, 3)$$

selects $g_1, A_0 g_2, g_3$. These vectors are linearly independent, and their selected tail ranks are $(3, 3, 2)$; hence the schedule is reachability-profile preserving.

By contrast,

$$\tau_{\text{bad}} = (1, 2, 3)$$

selects $A_0^2 g_1, A_0 g_2, g_3$. These vectors are again linearly independent, so the schedule is terminally controllable, but its selected tail-rank profile is

$$(3, 2, 1) \neq (3, 3, 2).$$

Equivalently, its block-count vector is $(1, 1, 1)$, rather than the required $(0, 1, 2)$. Thus terminal controllability does not imply preservation of the complete reachability profile.

10. AN EXPLICIT EXPONENTIAL-SEPARATION FAMILY

We now use the system of Section 9 to construct a family for which future-rank pruning reduces the number of nodes visited before the first solution from exponential to linear growth.

The counts below refer to the deterministic ordering used by the algorithm: candidate lists are ordered by increasing cardinality, ties preserve the original actuator order, and candidates within each list are examined in decreasing block index.

Proposition 10.1. *For $m \geq 1$, let*

$$A_m = I_m \otimes A_0, \quad \tilde{G}_m = I_m \otimes G_0,$$

and obtain G_m from $\tilde{G}_m$ by the column permutation

$$\pi_m = (1, 4, \dots, 3m-2, 2, 5, \dots, 3m-1, 3, 6, \dots, 3m).$$

Thus the actuators are ordered by type: first all type-1 actuators, then all type-2, and finally all type-3.

For horizon $N = 3$, set $B_j = A_m^{3-j} G_m$. Then

$$(w_1, w_2, w_3) = (3m, 3m, 2m), \quad (r_1, r_2, r_3) = (0, m, 2m).$$

With the ordering above, the baseline search visits

$$N_{\text{base}}(m) = 3 \cdot 2^m + m - 2$$

nodes before finding its first compatible basis, whereas the search with future-rank pruning visits

$$N_{\text{future}}(m) = 3m + 1.$$

Proof. Write

$$V = V_1 \oplus \dots \oplus V_m, \quad V_r = \text{Span}\{e_{r,1}, e_{r,2}, e_{r,3}\}.$$

In the r -th summand, the three actuator vectors and their images under A_m are

$$\begin{aligned} g_{r,1} &= e_{r,1}, & A_m g_{r,1} &= -e_{r,3}, \\ g_{r,2} &= e_{r,2}, & A_m g_{r,2} &= e_{r,1} + e_{r,3}, \\ g_{r,3} &= e_{r,1} + e_{r,2}, & A_m g_{r,3} &= e_{r,1}. \end{aligned}$$

The vectors in block 3 span $\text{Span}\{e_{r,1}, e_{r,2}\}$ in each summand, so $w_3 = 2m$. Adding block 2 supplies the missing direction $e_{r,3}$, hence $W_2 = V$ and $w_2 = 3m$. Also $w_1 = 3m$. Therefore

$$(w_1, w_2, w_3) = (3m, 3m, 2m), \quad (r_1, r_2, r_3) = (0, m, 2m),$$

and $k_0 = 2$.

We first consider the baseline search. Because the actuators are ordered by type, the first m representative families are of type 1. Their first candidates are the block-3 vectors $e_{r,1}$, which are mutually independent and belong to the branch leading to the first solution. Hence exactly one node is visited at each depth $0, \dots, m$.

For a type-2 actuator, the two candidates in the trimmed problem are

$$e_{r,2} \quad \text{from block 3}, \quad e_{r,1} + e_{r,3} \quad \text{from block 2}.$$

Both are independent of the previously selected $e_{r,1}$, and at this stage neither choice violates a block quota. Thus each type-2 actuator doubles the number of surviving branches. After s such actuators, there are 2^s nodes at depth $m + s$, so

$$N_k = 2^{k-m}, \quad m < k \leq 2m.$$

For a type-3 actuator, the candidates are

$$e_{r,1} + e_{r,2} \quad \text{from block 3,} \quad e_{r,1} \quad \text{from block 2.}$$

The block-2 candidate is always dependent on the previously selected type-1 vector $e_{r,1}$. The block-3 candidate is admissible exactly when the corresponding type-2 actuator was chosen from block 2. Indeed,

$$\{e_{r,1}, e_{r,2}, e_{r,1} + e_{r,2}\}$$

is dependent, whereas

$$\{e_{r,1}, e_{r,1} + e_{r,3}, e_{r,1} + e_{r,2}\}$$

is independent.

Each processed type-3 actuator therefore eliminates one half of the remaining branches. Hence

$$N_k = 2^{3m-k}, \quad 2m < k \leq 3m.$$

The complete depth profile is

$$N_k = \begin{cases} 1, & 0 \leq k \leq m, \\ 2^{k-m}, & m < k \leq 2m, \\ 2^{3m-k}, & 2m < k \leq 3m. \end{cases}$$

Summing over the depths gives

$$\begin{aligned} N_{\text{base}}(m) &= (m+1) + \sum_{s=1}^m 2^s + \sum_{s=1}^m 2^{m-s} \\ &= 3 \cdot 2^m + m - 2. \end{aligned}$$

Now consider future-rank pruning. Suppose that a type-2 actuator in V_r is tentatively chosen from block 3. The selected directions in this summand are then $e_{r,1}$ and $e_{r,2}$. The only remaining actuator is of type 3, whose possible vectors are $e_{r,1}$ and $e_{r,1} + e_{r,2}$. Thus every direction still available in V_r lies in $\text{Span}\{e_{r,1}, e_{r,2}\}$, so the attainable contribution of this summand to the level-2 rank is at most 2.

The remaining $m-1$ summands contribute at most $3(m-1)$ dimensions. Therefore the total attainable rank is at most

$$2 + 3(m-1) = 3m - 1 < w_2 = 3m.$$

The future-rank test rejects the block-3 choice immediately.

Consequently, every type-2 actuator is forced into block 2. After these choices, the type-3 block-3 candidates are independent of the previously selected vectors. The search therefore follows a single recursive branch through depths $0, \dots, 3m$, and hence

$$N_{\text{future}}(m) = 3m + 1.$$

□

Remark 10.1. A visited node means a call to the recursive DFS procedure. A candidate rejected by a quota, independence, or future-rank test does not create a new visited node.

Remark 10.2. The formulas in Proposition 10.1 refer to the fixed actuator and candidate ordering above and to the search for the first solution. They are not order-independent worst-case bounds. They exhibit a structured family for which future-rank pruning changes the first-solution search tree from exponential to linear size.

Since $d = 3m$, the same counts may be written as

$$N_{\text{base}}(d) = 3 \cdot 2^{d/3} + \frac{d}{3} - 2, \quad N_{\text{future}}(d) = d + 1.$$

Thus, for this family and the fixed search ordering, future-rank pruning changes the number of visited nodes before the first solution from exponential growth in d to linear growth.

11. COMPUTATIONAL EXPERIMENTS

We compare the baseline depth-first search with the version using future-rank pruning on the family of Section 10. All computations were carried out in exact arithmetic using the TailBasis package [7]. The package also contains the code for analyzing the examples and reproducing the computational experiments presented in this paper. A visited node means one call to the recursive depth-first search procedure.

For $m = 1, \dots, 8$, the observed numbers of visited nodes are shown in Table 1. Recall that the state-space dimension is $d = 3m$.

m	Baseline	Future-rank	Future-rank prunings
1	5	4	1
2	12	7	2
3	25	10	3
4	50	13	4
5	99	16	5
6	196	19	6
7	389	22	7
8	774	25	8

TABLE 1. Visited nodes and future-rank pruning events for the benchmark family.

The observed counts agree exactly with Proposition 10.1:

$$N_{\text{base}}(m) = 3 \cdot 2^m + m - 2, \quad N_{\text{future}}(m) = 3m + 1.$$

For example, at $m = 8$, hence $d = 24$, the baseline search visits 774 nodes, whereas the future-rank search visits 25.

The observed depth profiles also agree with the formulas derived in Section 10. For $m = 4$, the baseline search visits

$$1, 1, 1, 1, 1, 2, 4, 8, 16, 8, 4, 2, 1$$

nodes at successive depths, whereas the future-rank search visits one node at every depth. Thus the computations reproduce both the total node counts and the structure of the search tree predicted by Proposition 10.1.

12. FURTHER DIRECTIONS

The present formulation assumes d actuator channels in a d -dimensional state space and requires every actuator to be used exactly once. Several natural extensions remove these restrictions.

One may allow more actuator channels than state dimensions and seek both a subset of actuators and their activation times. A second possibility is to replace the single-use condition by prescribed upper bounds on the number of activations of each actuator. Both variants introduce additional combinatorial constraints into the representative selection problem.

Another direction is optimization. Once feasible reachability-profile preserving schedules are known to exist, one may minimize a prescribed cost depending on activation times, actuator usage, or control amplitudes. The present depth-first search can then serve as the basis of a branch-and-bound procedure.

13. CONCLUSION

We studied a constrained representative-selection problem for a block matrix $B = [B_1 | \cdots | B_n]$ through its rectangular tail matrices $B_{\geq k}$. Their column spaces form a nested family, while the rank differences $r_k = w_k - w_{k+1}$ determine the exact number of representatives that a compatible basis must select from each block. This reduces compatibility with the entire tail-space flag to a basis condition together with intrinsic block quotas.

For finite-horizon linear control, the same structure arises from the reachability matrix $[A^{N-1}G | \cdots | G]$. A single-use actuator schedule preserves the complete reachability profile precisely when its selected terminal directions form a compatible basis of representatives. This interpretation leads naturally to an exact depth-first search in which the tail ranks are used not only to determine the required block counts, but also to prune partial selections that can no longer attain the required future ranks.

For the explicit family constructed in Section 10, the number of visited nodes before the first solution changes from

$$N_{\text{base}}(d) = 3 \cdot 2^{d/3} + \frac{d}{3} - 2, \quad N_{\text{future}}(d) = d + 1.$$

The computational experiments reproduce these counts for all tested dimensions $d = 3, 6, \dots, 24$, illustrating that tail-rank information can substantially reduce the search tree when the underlying reachability structure is sufficiently rigid.

REFERENCES

- [1] L. Ballotta, G. Joseph, and I. Rahul Thete, Pointwise-sparse actuator scheduling for linear systems with controllability guarantee, *IEEE Control Systems Letters* **8** (2024), 2361–2366. doi: 10.1109/LCSYS.2024.3475886.
- [2] J. Edmonds and D. R. Fulkerson, Transversals and matroid partition, *Journal of Research of the National Bureau of Standards, Section B: Mathematics and Mathematical Physics* **69B** (1965), no. 3, 147–153. doi: 10.6028/jres.069B.016.
- [3] P. Hall, On representatives of subsets, *Journal of the London Mathematical Society* **s1-10** (1935), no. 1, 26–30. doi: 10.1112/jlms/s1-10.37.26.
- [4] G. Joseph, Sparse actuator control of discrete-time linear dynamical systems, *Foundations and Trends in Systems and Control* **11** (2024), no. 3, 186–284. doi: 10.1561/26000000033.
- [5] K. P. V. S. Kondapi, C. Sriram, G. Joseph, and C. R. Murthy, Sparse actuator scheduling for discrete-time linear dynamical systems, in *Proceedings of the 2024 10th Indian Control Conference (ICC)*, IEEE, 2024, pp. 84–89. doi: 10.1109/ICC64753.2024.10883726.
- [6] K. P. V. S. Kondapi, C. Sriram, G. Joseph, and C. R. Murthy, Discrete-time linear dynamical system control using sparse inputs with time-varying support, arXiv:2506.18514, 2025.
- [7] D. E. Kędziński and P. Krasoń, *TailBasis: Compatible Bases and Actuator Scheduling*, version 0.1.0, Zenodo, 2026. doi: 10.5281/zenodo.23102973.
- [8] J. Oxley, *Matroid Theory*, 2nd ed., Oxford University Press, Oxford, 2011. doi: 10.1093/acprof:oso/9780198566946.001.0001.
- [9] R. Rado, A theorem on independence relations, *The Quarterly Journal of Mathematics* **os-13** (1942), no. 1, 83–89. doi: 10.1093/qmath/os-13.1.83.
- [10] A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency*, Algorithms and Combinatorics, vol. 24, Springer, Berlin, 2003.
- [11] E. D. Sontag, *Mathematical Control Theory: Deterministic Finite Dimensional Systems*, 2nd ed., Texts in Applied Mathematics, vol. 6, Springer, New York, 1998. doi: 10.1007/978-1-4612-0577-7.
- [12] T. H. Summers, F. L. Cortesi, and J. Lygeros, On submodularity and controllability in complex dynamical networks, *IEEE Transactions on Control of Network Systems* **3** (2016), no. 1, 91–101. doi: 10.1109/TCNS.2015.2453711.

INSTITUTE OF MATHEMATICS, UNIVERSITY OF SZCZECIN,, UL. WIELKOPOLSKA 15,
70-451 SZCZECIN, POLAND,
Email address: dawidedmundkedziński@gmail.com

DEPARTMENT OF MATHEMATICS, ARIEL UNIVERSITY, ARIEL 40700, ISRAEL
Email address: piotrkras26@gmail.com